

CSCD602:

Advanced Software Engineering

Session 2 – Requirements Engineering

By

Solomon Mensah (PhD)

Dept. of Computer Science

smensah03@ug.edu.gh



UNIVERSITY OF GHANA

Session Outline

The key topics to be covered in the session are as follows:

- Introduction to Requirements Engineering
- Types of Requirements
- Advanced Requirements Elicitation Techniques
- Functional vs Non-Functional Requirements
- Change Management

Learning Outcomes

At the end of the session, you will be able to ;

- Explain the role and importance of Requirements Engineering
- Apply advanced elicitation techniques to gather accurate system requirements
- Differentiate between functional and non-functional requirements with practical examples

Requirements Engineering

- Requirements Engineering (RE) is the systematic process of defining, documenting, and maintaining stakeholder needs for a system or software product.
- It involves eliciting, analyzing, specifying, and validating requirements to ensure they are clear, achievable, and aligned with project goals, ultimately preventing project failures.
- It forms the foundation of software and systems development. Errors made during requirements engineering are among the most expensive and difficult to fix later in the lifecycle.

Scope of Requirements Engineering

The scope of RE spans the entire lifecycle, from initial concept to system maintenance, involving several core activities:

- Requirements Elicitation:
 - Gathering needs from stakeholders via interviews, surveys, workshops, and observation.
- Requirements Analysis & Negotiation:
 - Evaluating, prioritizing, and resolving conflicts to ensure requirements are feasible, complete, and consistent.

Scope of Requirements Engineering

- Requirements Specification (Documentation):
 - Clearly recording requirements in documents like SRS (Software Requirements Specification), using use cases and user stories.
- Requirements Validation & Verification:
 - Checking that documented requirements accurately meet stakeholder needs and are technically sound.
- Requirements Management:
 - Handling changes, traceability, and maintenance of requirements throughout the project lifecycle

Types of Requirements

The scope covers various levels of detail and perspectives:

- **Business Requirements:** High-level goals and objectives of the organization.
- **Stakeholder Requirements:** Specific needs and expectations of various user groups.
- **Functional Requirements:** Descriptions of what the system must do (e.g., behaviors, operations).
- **Non-Functional Requirements:** Quality attributes such as performance, security, usability, and reliability.
- **Constraints:** Regulatory, legal, or technical limitations on the system.

Examples for Types of Requirements

1. Business Requirements

- **High-level goals and objectives of the organization**
 - The university wants to allow students to register for courses online.
 - The system should reduce long queues during registration.
 - The university aims to improve registration accuracy and reduce paperwork.
 - The system should support at least 5,000 students per semester.
- These describe *why* the system is being built.

Examples for Types of Requirements

2. Stakeholder Requirements

- **Specific needs and expectations of different user groups**
 - Students want to register for courses easily using their phones or laptops.
 - Lecturers want to see the list of students registered for their courses.
 - Administrators want to approve course registrations and generate reports.
 - IT Department wants the system to integrate with the existing student database.
- These describe what different users expect.

Examples for Types of Requirements

3. Functional Requirements

- **What the system must do**
 - The system shall allow students to log in using their student ID and password.
 - The system shall display available courses for each semester.
 - The system shall allow students to add or drop courses.
 - The system shall generate a registration confirmation slip.
 - The system shall allow administrators to approve or reject course selections.
- These describe system behaviors and operations.

Examples for Types of Requirements

4. Non-Functional Requirements

- **Quality attributes of the system**
 - The system should load each page within 3 seconds.
 - The system should be available 99% of the time.
 - The system should protect user data using encryption.
 - The system should be easy to use with a simple interface.
 - The system should support up to 1,000 users at the same time.
- These describe how well the system performs.

Examples for Types of Requirements

5. Constraints

- **Limitations on the system**
 - The system must comply with Ghana Data Protection laws.
 - The project budget must not exceed GHS 200,000.
 - The system must use the university's existing database server.
 - The system must be completed within 6 months.
 - The system must run on both Windows and Linux servers.
- These describe restrictions or limitations.

Benefits of Requirements Engineering

- **Direction and Structure:** Provides clear guidance on what to develop, keeping teams focused and reducing uncertainty.
- **Risk Mitigation:** Identifies potential issues early, preventing costly and time-consuming fixes later.
- **Stakeholder Alignment:** Ensures all parties share a common understanding of scope and outcomes, minimizing misunderstandings.
- **Improved Quality and Performance:** Leads to software that meets user needs, reduces defects, and enhances stability and usability.

Advanced Requirements Elicitation Techniques

Why “Advanced” Elicitation is Necessary?

- Basic elicitation often fails because:
 - Stakeholders do not fully understand their needs
 - Users describe *solutions* instead of *problems*
 - Requirements are tacit (not clearly stated)
 - Different stakeholders have conflicting goals
- Advanced techniques address complexity, ambiguity, and human factors.
 - Requirements elicitation is the process of gathering, discovering, and identifying what users and stakeholders need from a system.
 - It is the process of asking the right questions to understand what the system should do and what problems it should solve.

Interviews (Advanced Perspective)

Types of Interviews

- **Structured interviews:** predefined questions
- **Semi-structured interviews:** flexible questioning
- **Unstructured interviews:** open discussion

Strengths

- Allows deep exploration of domain knowledge
- Clarifies vague or contradictory statements

Weaknesses

- Time-consuming
- Subject to interviewer bias

Workshops & Joint Application Development (JAD)

Facilitated sessions where stakeholders collaboratively define requirements.

Benefits

- Encourages consensus building
- Reduces conflicts early
- Improves shared understanding

Challenges

- Dominant personalities may suppress others
- Requires skilled facilitation

Example

ERP System

- Finance, HR, procurement, and IT teams jointly define system workflows

Observation & Ethnography

Observation is often ethnographic, meaning:

- Analysts immerse themselves in user environment
- Focus on *actual behavior*, not stated behavior

Why This Matters?

- Users often skip steps
- Create workarounds
- Use systems differently than intended

Example

- **Hospital Information System**
 - Nurses write notes on paper despite digital system

Prototyping in Requirements Engineering

Prototyping is the process of creating early models of a system before it is fully developed.

Purpose:

- To explore ideas
- Gather user feedback
- Identify missing or misunderstood requirements
- Validate design choices before expensive development

Prototyping is an essential part of requirements elicitation, because users often cannot fully articulate their needs in words.

Low-Fidelity (Lo-Fi) Prototypes

A low-fidelity (lo-fi) prototype is a basic, early-stage, tangible representation of a product concept, focusing on core structure, flow, and functionality over visual details, often created quickly with sketches, wireframes, or simple digital tools for rapid testing and feedback on fundamental ideas before investing in detailed design.

Key Characteristics

- **Simplicity:** Minimal design, often black and white, with placeholder content, highlighting layout and hierarchy.
- **Functionality Focus:** Emphasizes core purpose and user flow over final visuals.
- **Speed:** Fast and cheap to make using paper, whiteboards, or basic digital tools like Figma/Miro.
- **Interactivity:** Can be static screens or basic linked wireframes, with humans simulating responses.

Types of Lo-Fi Prototypes

1. Paper Sketches

- Drawn on paper or whiteboard
- Represent screens, workflows, or user interfaces
- Often used in brainstorming sessions
- Quick to create, easy to change

Advantages:

- Very fast to make
- Requires minimal resources
- Encourages creativity and experimentation

Types of Lo-Fi Prototypes

2. Wireframes

- More structured than sketches
- Shows layout, content, and navigation
- Usually created with tools like Figma, or Adobe XD

Advantages:

- Clearer representation of UI structure
- Can indicate workflow and interaction
- Still quick and inexpensive

Limitations:

- Lacks real interactivity
- Users cannot experience realistic system behavior

High-Fidelity (Hi-Fi) Prototypes

- High-fidelity prototypes are interactive, detailed, and realistic versions of the system.
- They simulate actual system behavior as closely as possible without being the final product.

Types of Hi-Fi Prototypes

1. Interactive Mockups

- Created with prototyping tools like Figma, Adobe XD, Sketch, or InVision
- Users can click through screens and workflows
- Looks like the real application

Advantages:

- Provides realistic experience
- Can test usability and workflows
- Allows stakeholders to understand exact behavior

High-Fidelity (Hi-Fi) Prototypes

2. Near-Real UI Behavior

- Simulates system logic, navigation, animations, and feedback
- Can sometimes connect to mock data to feel dynamic

Advantages:

- Identifies usability issues early
- Reduces rework during development
- Encourages early user buy-in

Limitations:

- Time-consuming to create
- Requires more skills and tools

Role of Prototypes in Requirements Engineering

a) Uncover Missing or Misunderstood Requirements

- Lo-fi prototypes reveal conceptual gaps early
- Hi-fi prototypes expose workflow errors or UI issues

Example:

- Paper prototype shows a “Request Ride” button but forgets a “Cancel Ride” option
- Stakeholders notice this early, avoiding costly changes later

Role of Prototypes in Requirements Engineering

b) Early Validation

- Stakeholders and end-users can interact with the system before development
- Developers can confirm assumptions about how the system should work

- **Example:**

- Hi-fi ride-hailing prototype allows users to simulate booking a ride
- Feedback shows that location selection is confusing
- Adjustments can be made before coding begins

Functional vs Non-Functional Requirements

Functional Requirements define what a system does (features, actions, tasks like login, payment), functional requirements describe behavior (e.g., "system shall send email")

Functional requirements specify:

- System services
- Business rules
- System reactions to inputs

They are often:

- Expressed as use cases
- Mapped to business processes

Example (E-Commerce)

- Calculate total cost including tax and shipping
- Apply discount rules

Non-Functional Requirements (Quality Attributes)

Non-functional requirements define constraints on system behavior.

- They are often more critical than functional requirements because:
 - They affect user satisfaction
 - They impact architecture decisions

They ensure the system is efficient, secure, and maintainable over time.

Sample Questions

- How fast should the system respond to user actions?
- How secure should it be against unauthorized access?
- How available and reliable should the system be?

Example of Functional and Non-functional Requirements

Online Banking System

Functional Requirements:

- Users should be able to log in with their username and password.
- Users should be able to check their account balance.
- Users should receive notifications after making a transaction.

Non-functional Requirements:

- The system should respond to user actions in less than 2 seconds.
- All transactions must be encrypted and comply with industry security standards.
- The system should be able to handle 100 million users with minimal downtime.

Change Management

Change management is the structured process used to control and manage changes to software requirements in a dynamic environment.

Why Changes are Inevitable?

- Software requirements change due to evolving internal and external factors.

1. Market Changes

- Businesses adapt to customer needs and competition.

Example: E-commerce platforms add new payment options.

2. User Feedback

- Users request improvements or new features after release.

Example: Social media apps update features based on user behavior.

Change Management

Why Changes are Inevitable?

3. New Regulations

- Legal and regulatory requirements force system updates.
Example: GDPR requires stronger data privacy controls.

4. Technological Evolution

- New technologies require system upgrades or migration.
Example: Moving from desktop applications to cloud systems.

Formal Change Control Process

Change control ensures changes are properly evaluated and implemented.

- **Change Request Logging**
 - A formal request describing the change, reason, and benefits is documented.
- **Impact Analysis**
 - Assesses effects on architecture, code, testing, cost, and schedule using traceability.
- **Cost, Schedule, and Risk Assessment**
 - Evaluates financial cost, implementation time, and potential risks.
- **Change Control Board (CCB) Approval**
 - A committee decides to approve, reject, or modify the change.
- **Implementation**
 - Approved changes are applied to design, code, documentation, and testing.

Capstone Project Topic Task

- Identify your group members and meet to discuss your proposed capstone project topic
- Prepare an SRS with an AI assistive tool like Chatgpt, claud, etc for the chosen and agreed topic
- Use any AI assistive tool like Figma Make to develop a prototype for presentation to the class
- Note that you will also need to present the prompt(s) used for both the SRS and the prototype

Thank you

